Tiny-YOLOSAM: Fast Hybrid Image Segmentation

Kenneth Xu[1], **Mason Wu**[1], **Abby Schultz**[1], **Katherine Zhang**[1], **Max Han**[1]

[1]University of Michigan
{kennethx, wuumaa, abigayls, kathyzz, maxhan}@umich.edu
Group 1

Abstract

Foundational vision models like Meta's Segment Anything Model (SAM) have revolutionized image segmentation through zero-shot generalization; however, their substantial computational requirements make them impractical for use on consumer-grade hardware. While distilled variants such as TinySAM address model size and modern object detectors like YOLOv12 address speed, relying solely on TinySAM's default *segment everything* mode still creates a significant bottleneck due to its computationally expensive grid sampling strategy. This research initially performs a full replication of both TinySAM and YOLOv12, successfully reproducing benchmark metrics including mAP, GFLOPs, latency, etc. This validation confirms that while TinySAM is lightweight, its hierarchical sampling method remains too slow for practical application, taking upwards of 40 seconds per image to generate comprehensive masks.

To bridge the gap between quality image segmentation and speedy processing, we propose and evaluate *Tiny-YOLOSAM,* a hybrid pipeline that replaces dense grid prompting with object aware guidance. In this architecture, YOLOv12 is first used to rapidly generate bounding box proposals, which serve as prompts for the TinySAM mask decoder. Object detectors often miss background elements (such as the sky or roads), so we acknowledge this weakness by integrating a supplementary sparse point grid. This hybrid design uses YOLO's speediness for foreground objects while using extra background sampling to ensure that everything is accounted for, effectively eliminating the redundancy of standard hierarchical decoding.

Our experimental results demonstrate that this pipelined approach significantly outperforms the baseline in terms of efficiency while maintaining competitive segmentation quality. The hybrid pipeline is approximately 4˜5 times faster than TinySAM's native hierarchical mode, reducing inference time from approximately 49 seconds to 10 seconds per image. The addition of sparse background sampling also dramatically improves Average Recall (AR) to 77.1%. While there is a minor tradeoff in precision for smaller objects, our findings present a viable pathway for achieving high quality, whole image segmentation on consumer grade hardware by combining specialized models.

Introduction

TinySAM, as its name implies, is a smaller, distilled version of the original Segment Anything Model (SAM). While the original SAM model is extremely powerful and capable of producing pixel-level segmentation masks, it has two major problems: it is extremely slow and extremely resource intensive (especially in its "segment everything" mode). The key results of TinySAM were that it could achieve higher zero-shot segmentation accuracy than other SAM variants, while being extremely lightweight and having a computational cost smaller by several magnitudes.

Similarly, speed was a key strength of the original YOLO algorithm. YOLO was not only the first single-shot object detection algorithm but also extremely fast, to the point of completely dominating earlier detectors. As later versions came out, it was expanded to more scenarios (video, pose estimation, etc.), and its latest version, YOLOv12, further improved its efficiency and speed-accuracy tradeoff. Because detection models are often deployed in settings where memory and latency constraints matter (such as mobile devices, robotics, and real-time perception), these improvements are of practical importance, not just academic interest.

Since both of these papers focused on improving the efficiency, speed, and compactness of their respective models, we were particularly interested in replicating and verifying the models' performance according to these metrics. In this work, we first reproduce the core results of TinySAM and YOLOv12 to ensure a trustworthy experimental foundation. This replication step also allows us to evaluate their behavior under consistent hardware and evaluation settings, providing a fair basis for the extension we later develop.

While TinySAM and YOLOv12 are powerful, highly influential models in the field of image segmentation, as with all models, they must make tradeoffs between accuracy, granularity, and computational efficiency. TinySAM enables fine-grained, pixel-level segmentation with its generation of image masks. However, it is reliant on external prompts such as text queries or bounding boxes to achieve practical run times. Without prompting, its "segment everything" mode is still too slow to be practical for real-world applications in conditions where time and resources are limited. In contrast, YOLOv12 provides high-throughput object detection at a relatively low computational cost. However, its output is limited to coarse bounding box segmentation, and at times suffers from incomplete object coverage.

In our extension, we aim to address their respective limitations through a hybrid approach, in which the bound-

ing boxes generated by YOLOv12 are used to prompt TinySAM. This coupling leverages YOLO's efficiency and TinySAM's granularity to achieve an optimal middle ground, enabling fast generation of segmentation masks. To ensure full-scene coverage, we additionally incorporate a lightweight sparse point graph, which supplements YOLO's detections by prompting TinySAM on missing or background regions.

Throughout this paper, we demonstrate that our hybrid model combining YOLO with TinySAM, supported by sparse point prompting, yields segmentation quality comparable to the original TinySAM pipeline at a dramatically higher speed. These results highlight how detector-guided prompting can serve as an effective alternative to dense grid-based "segment everything" methods, enabling efficient and practical full-scene segmentation on consumer-grade hardware. Our code can be found here: https://github.com/Kenneth-Xu11566/tiny-yolosam

Related Work

1. MobileSAMv2: In SAM/MobileSAM's segment-everything mode, the primary bottleneck is the mask decoder because of the dense grid it uses to generate point prompts. MobileSAMv2 replaced this dense grid sampling strategy with YOLOv8's box prompts. This new object-aware sampling significantly decreased the number of prompts needed to be processed in the mask decoder, yielding a 16x speedup. This contributed a massive improvement to the efficiency of the segment-everything task. The primary limitation of this method was that the model becomes dependent on YOLOv8's performance. The dense grid method, while exhaustive, was good at achieving full coverage. While for the YOLO strategy, if YOLOv8 failed to identify a smaller object, it would be missed entirely by SAM. Our extension, aside from using TinySAM for maximum efficiency and compactness, also provides a sparse point graph to improve coverage while maintaining high efficiency. (Zhang et al. 2023a)
2. EdgeSAM: EdgeSAM distills SAM into a CNN-based architecture with the goal of allowing it to run effectively on edge devices. It completely swaps out the ViT encoder with a CNN encoder. It also uses a "prompt-in-the-loop" distillation strategy, where SAM acts as a teacher, and new prompts are introduced iteratively where the student model exhibits inaccuracies. This approach is reportedly 37x faster than SAM and 7x faster than MobileSAM. Similar to TinySAM and the other papers, its main goal is to try and distill the SAM model to be more practical for real-time applications, on more diverse devices (in this case edge devices). However, this paper has a few major limitations, one of which happens to be addressed by our extension as well as in MobileSAMv2. It still uses the dense grid method in the mask decoder, meaning the segment-everything mode is still way too expensive for deployment on the edge-devices the paper is optimizing for. Another limitation is that it is reliant on SAM as a teacher during training, which is again a very large and resource intensive model. (Zhang et al. 2023b)
3. FastSAM: FastSAM, while having 'SAM' in the name, is less so a version/improvement of SAM and more so a drop-in replacement (with lower accuracy but higher efficiency). It's a real-time alternative that replaces the SAM model, in which all-instance segmentation is performed by a CNN-based detector (in this case YOLOv8-seg), then uses user prompts to select masks of interest. While this was a very strong alternative to SAM when it originally came out, it was significantly beaten by by TinySAM, which had both higher performance and efficiency. However, with more modern versions of YOLO-seg like YOLOv12-seg, it could close this gap significantly. Unfortunately, we weren't able to find a paper for FastSAMv2. However, this paper still demonstrated general strengths and weaknesses of the YOLO model that informed considerations for our extension. YOLO originated as an object detector. Therefore, it is far better suited for segmenting discrete object instances and tends to struggle for things in the background, or things that are not a traditional object (for example, a city-scape or regions of sky). In our extension, we improve upon this weakness with a sparse grid, to make sure such non-traditional objects can be better represented. (Zhao et al. 2023)

Background

Image Segmentation and SAM

Image segmentation partitions images into meaningful regions. The Segment Anything Model (SAM) (Kirillov et al. 2023) introduced prompt-guided universal segmentation, where users specify regions via points, boxes, or masks (further developed to texts, audio, etc.). SAM uses a Vision Transformer (ViT) encoder and lightweight mask decoder to produce high-quality masks with zero-shot generalization. However, SAM's large encoder (632M parameters) results in slow inference, limiting real-time applications.

TinySAM

TinySAM (Shu et al. 2023) addresses SAM's computational cost through three techniques: (1) *Hard Mining Full-Stage Knowledge Distillation (Appendix A)* from SAM's ViT encoder (teacher) to a smaller TinyViT encoder (student), (2) *Quantization (Appendix B)* to 8-bit integers, reducing model size by 4×, and (3) *Hierarchical Segment-Everything (Appendix C)*, first generating masks from a sparse 8×8 grid, then applying dense sampling to regions not covered by high-confidence predictions. Despite achieving 42.3% AP on COCO with significant speedup over SAM, TinySAM still requires dense prompting (hundreds of points per image), its *hierarchical segment-everything* mode would take 31-44 seconds per image in our experiments.

YOLO for Object Detection

The YOLO (You Only Look Once) family performs single-stage object detection, directly predicting bounding boxes and classes in one forward pass. YOLOv12 (Yunjie, Qixiang, and David 2024) introduces Transformer backbones

with *Area Attention (Appendix D)* and *Flash Attention (Appendix E)* for efficient memory access. YOLOv12s achieves 47.6% mAP on COCO with only 3.1ms latency. While YOLO excels at detecting prominent objects, it only outputs coarse bounding boxes and may miss objects outside its training categories.

Evaluation Metrics

We use COCO 2017 val split for evaluation. (Lin et al. 2014) Key metrics include: *Average Precision (AP)* at IoU 0.50:0.95 for instance segmentation, further subdivided by object size: small, medium, and large; *Average Recall (AR)* measuring coverage; *Mean IoU (mIoU)* for mask quality. Other than precision metrics provided by COCO, we use *inference time* (seconds/image) and *calls of decoder* to measure the models' speed.

Methodology, Results, and Discussion

TinySAM Replication

We first replicate TinySAM's baseline performance to verify the reliability of our experimental environment. Using official pretrained weights (both non-quantized and quantized models), we evaluate on the full COCO val2017 dataset (5000 images). Hardware: Apple M1 Pro Silicon.

Table 1: TinySAM Replication Results on COCO val2017

Model	AP@0.50:0.95	Paper Claim	Error
TinySAM (Non-Quantized)	42.27%	42.3%	0.03%
TinySAM (Quantized W8A8)	41.37%	41.4%	0.03%

Error within 0.03%, we successfully confirmed the correctness of our experimental setup and evaluation pipeline.

YOLOv12 Replication

We replicate the performance comparison between YOLOv12 and prior YOLO models to verify its advantages in the speed–accuracy trade-off. All models were evaluated using their official pretrained weights on the COCO val2017 dataset under consistent software and hardware conditions. The replication environment consisted of Ultralytics 8.3.63, PyTorch 2.4.1+cu121, and Python 3.11.14, and all experiments were conducted on an NVIDIA RTX 3060 GPU.

YOLOv12 achieves the best accuracy-speed balance, significantly improving detection accuracy while maintaining low latency compared to previous models. Main technical innovations include: (1) Transformer replacing CNN backbone for enhanced modeling capability; (2) Area Attention module reducing self-attention complexity from $O(L^2d)$ to $O(L^2d/16)$; (3) Residual Efficient Layer *(Appendix F)* Aggregation; (4) Flash Attention optimizing memory access efficiency.

Table 2: YOLOv12 vs Previous YOLO Models

Model	GFLOPs	#Params(M)	AP@0.50:0.95	Latency(ms)
YOLOv12-N	6.0	2.5	40.3%	1.6
YOLOv11-N	2.6	6.5	39.4%	2.1
YOLOv10-N	6.7	2.3	38.3%	1.4
YOLOv8-N	8.7	3.2	37.1%	1.3
YOLOv6-3.0-N	11.4	4.7	37.5%	2.3
YOLOv12-S	**19.4**	**9.1**	**47.6%**	**3.1**
YOLOv11-S	21.5	9.4	46.9%	4.2
YOLOv10-S	21.6	7.2	46.2%	2.6
YOLOv9-S	26.9	7.2	46.4%	3.2
YOLOv8-S	28.6	11.2	44.8%	2.4
YOLOv6-3.0-S	45.3	18.5	45%	5.9
YOLOv12-M	**59.8**	**20.0**	**52.5%**	**6.1**
YOLOv11-M	68.0	20.1	55.7%	10.0
YOLOv10-M	59.1	15.4	51.1%	5.4
YOLOv9-M	76.8	20.1	51.4%	6.4
YOLOv8-M	78.9	25.9	50.2%	5.6
YOLOv6-3.0-M	85.8	34.9	50%	12.4
YOLOv12-L	**82.4**	**26.4**	**53.8%**	**8.6**
YOLOv11-L	86.9.0	25.3	53.3%	12.8
YOLOv10-L	120.3	24.4	53.2%	8.6
YOLOv9-C	102.7	25.4	52.8%	8.1
YOLOv8-L	165.2	43.7	53%	9.0
YOLOv6-3.0-L	150.73	59.6	52.8%	19.9
YOLOv12-X	**184.6**	**59.3**	**55.4%**	**15.6**
YOLOv11-X	194.9	56.9	54.6%	24.6
YOLOv10-X	160.4	29.5	54.5%	12.8
YOLOv8-X	257.8	68.2	54.0%	14.7

Hierarchical Segment-Everything Baseline

TinySAM's original "segment everything" mode employs two-stage hierarchical point sampling: first sampling 64 points on an 8×8 coarse grid, then adaptively sampling 469 points in low-confidence regions, totaling 533 points.

Key performance characteristics include a processing time of 2.89 seconds per image, 13.1 TinySAM calls per image, and coverage metrics of AR=16.4% and mIoU=19.2%.

Notes on Hardware and Batching: Hierarchical baseline performance varies significantly across platforms: 49.20s on M1 Pro (CPU) vs. 2.89s on RTX 4070 (GPU with batching). We report both values for transparency, using the M1 Pro measurements as our primary comparison baseline to maintain consistency with hybrid system evaluation.

While this method achieves full-image segmentation, it suffers from two critical issues: (1) extremely expensive computational overhead, unsuitable for practical applications; (2) poor coverage and segmentation quality, missing many ground truth objects.

Extensions

Full-scene segmentation is slow because models like SAM and TinySAM must evaluate hundreds of prompts across the whole image. MobileSAMv2 (Zhang et al. 2023a) showed that using a detector to guide these prompts can make segmentation dramatically more efficient. Building on this idea, we investigate whether pairing YOLO with TinySAM can create a lightweight, fast, and accurate "segment everything" pipeline.

Naive Extension: YOLO→TinySAM

Our first extension explores a straightforward integration of YOLOv12 and TinySAM, leveraging the strengths of each model to accelerate instance segmentation while retaining high mask quality. The central idea is to replace TinySAM's expensive hierarchical "segment everything" procedure with objectness-aware box prompts supplied directly by YOLO. This enables TinySAM to focus exclusively on the most salient regions of an image, dramatically reducing the number of prompts and decoder calls required for mask generation.

The naive pipeline consists of three primary stages. First, YOLOv12s-Turbo processes the input image and outputs a set of bounding boxes, class labels, and confidence scores. Second, each YOLO detection box is converted into a box prompt and passed to TinySAM's mask decoder, which produces a high-resolution segmentation mask for the detected object. Since TinySAM excels at refining object outlines within a spatially constrained region, this step provides detailed, pixel-accurate masks without requiring dense sampling of the entire image. Finally, duplicate masks are removed using IoU-based non-maximum suppression, and the resulting instance masks form the complete segmentation output.

Detector-guided prompting offers several advantages over TinySAM's hierarchical segment-everything baseline. Most notably, it provides a 7.6× speedup (0.38s vs. 2.89s per image) and dramatically improved mask quality (mIoU 0.93 vs. 0.19) for those masks that are comparable, while requiring only 8.1 decoder calls compared to the hierarchical method's 13.1.

Table 3: Hierarchical v.s. YOLO→TinySAM Comparison

System	mIoU	Time/img	Calls/img
Hierarchical	0.19	2.89	13.1
YOLO→TinySAM	**0.93**	**0.38**	**8.1**

Unlike the core TinySAM paper, which relies on coarse-to-fine refinement across the entire image, our method uses YOLO to reduce redundant sampling by targeting only semantically meaningful regions. Compared to MobileSAMv2, which uses YOLOv8 box prompts to accelerate SAM, our approach leverages YOLOv12's stronger detection performance and pairs it with TinySAM's compact decoder to achieve a more favorable efficiency–accuracy tradeoff for real-time usage.

These results demonstrate that detector-guided prompting is a practical and highly efficient alternative to hierarchical sampling, establishing a YOLO to TinySAM pipeline as a strong baseline for real-time, instance-aware segmentation. However, this approach is fundamentally limited by YOLO's recall: it segments only COCO object categories (˜80 species) and completely misses background regions such as the sky, terrain, or surrounding buildings. This motivates our hybrid extension, which incorporates sparse point prompts to recover background coverage and achieve full-scene segmentation.

Hybrid Extension: YOLO + Sparse Points

After evaluating the limitations of the YOLO→TinySAM pipeline, particularly its inability to segment background or non-COCO categories, we introduce a hybrid approach designed to recover full-scene segmentation while retaining detector-guided efficiency. The core idea is to let YOLO handle salient foreground objects, while delegating the remaining unsegmented regions of the image to TinySAM through sparse point prompts.

Our hybrid method begins with YOLOv12 detecting prominent foreground objects, which are segmented by TinySAM using box prompts. We then construct a binary coverage mask that marks all pixels already explained by these foreground masks. Instead of sampling dense grids, we place a lightweight 16×16 sparse grid only over the uncovered regions, producing roughly 80–150 point prompts depending on scene complexity. These points prompt TinySAM to generate background and miscellaneous object masks that YOLO cannot detect. Finally, we merge the foreground and background masks by removing duplicate or heavily overlapping masks using an IoU threshold, ensuring that each region of the image is assigned a single, clean segmentation. This pipeline preserves the speed and semantic precision of the detector while recovering the comprehensive coverage characteristic of segment-everything systems, showing inspirations inherited from the original hierarchical segment-everything design *(Appendix H)*.

Quantitatively, the hybrid approach substantially improves coverage compared to both Naive(YOLO-only) and TinySAM's hierarchical baseline.

Before we delve into the evaluation, it is necessary to make it clear why we divide our extension's evaluation metrics into two parts: Detector-based and Class-agnostic. Because `category_id` is a necessary parameter for COCO evaluation, original TinySAM manually added ViTDet in the pre-processing stage to provide them (so original TinySAM's images come with box prompts and their `category_id` provided by ViTDet). In our extension, for foreground objects with YOLO bounding boxes, we have `category_id` inherently, making COCO metrics feasible. Thus, this "Detector-based" part is evaluated by *AP*. However, for background masks generated by sparse points, we don't have the `category_id`, we choose to use Class-agnostic metrics, such as *AR* and *mIoU*.

On 500 COCO val2017 images, for the Class-agnostic objects, average recall increases from 16.4% to 77.1% (4.7× improvement), and mIoU rises from 19.2% to 67.8% (3.5× improvement). Although the hybrid system is slower than the YOLO-only pipeline due to additional TinySAM calls (about 174 calls per image), it still achieves a 4.7× speedup over the hierarchical baseline (10.39s vs. 49.20s). In Detector-based AP metrics, the hybrid system also demonstrates efficiency gains, achieving a 4.3× speedup over ViTDet→TinySAM (0.38s vs. 1.63s) with fewer decoder calls (8.1 vs. 17.6). However, YOLO→TinySAM's *AP* remains 6 percentage points below the ViTDet→TinySAM baseline, primarily because YOLOv12 misses 38.1% of small or infrequent objects. Because these objects have already been

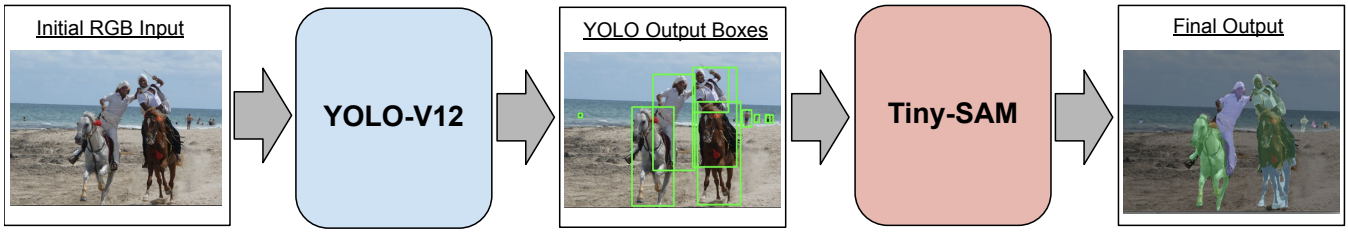


Figure 1: Pipeline for the naive YOLO→TinySAM extension. YOLOv12 first detects objects and outputs bounding boxes. Each bounding box is then used as a box prompt for TinySAM, which generates instance segmentation masks for detected objects.

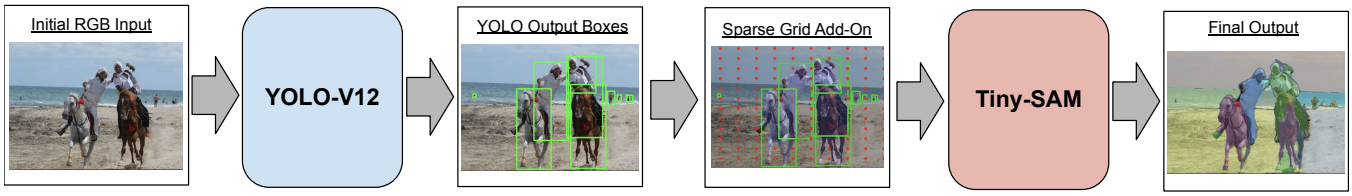


Figure 2: Pipeline for the hybrid YOLO + sparse points extension. YOLOv12 first detects salient foreground objects, which are segmented by TinySAM using box prompts. A coverage mask is then constructed to mark regions already segmented. Sparse points are sampled only in the remaining uncovered regions and passed to TinySAM as point prompts to recover background and small objects. All masks are finally merged using IoU-based post-processing to form full-scene segmentation.

included (yet not detected) in the 'YOLO regions', they will also not be exposed to later background sparse grid detection.

Table 4: Detector-based Evaluation (COCO Instance Segmentation Metrics)

System	AP	AP_s	AP_m	AP_l
ViTDet→TinySAM	46.7%	28.9%	49.7%	59.4%
Hybrid (boxes only)	**40.7%**	**17.6%**	**44.8%**	**50.0%**

Table 5: Detector-based Evaluation (Speed Metrics)

System	Time/img	Calls/img
ViTDet→TinySAM	1.63	17.6
Hybrid (boxes only)	**0.38**	**8.1**

Table 6: Class-agnostic Evaluation (Coverage-focused Metrics)

System	AR	mIoU	Time/img	Calls/img
Hierarchical	16.4%	19.2%	49.20	13.1
Hybrid	**77.1%**	**67.8%**	**10.39**	**174.2**

Notes: Visualized Graph of Table 4, 5, 6 in Appendix I

On the other hand, Qualitative results illustrate these gains clearly. In a beach horse-riding scene, the hierarchical baseline produces numerous fragmented masks, while the YOLO-only pipeline captures only the horses and people, ignoring most of the environment. The hybrid method, in contrast, segments both the foreground subjects and large background areas such as the ocean and shoreline, achieving substantially higher recall rate and visual completeness.



Figure 3: Visual comparison of segmentation quality across the three systems evaluated in our study. **Left: Hierarchical TinySAM** produces coarse and fragmented masks due to dense but low-confidence sampling. **Middle: YOLO-only** captures prominent foreground objects but fails to segment background regions or smaller structures. **Right: Hybrid (YOLO + Sparse Points)** recovers both foreground and background content, producing substantially more complete scene coverage. These examples highlight the main advantage of our extension: combining fast detector-guided prompts with sparse, targeted sampling yields both higher coverage and more coherent full-scene segmentation.

Discussion

Our results highlight both the strengths and limitations of pairing YOLOv12 with TinySAM for fast, full-scene segmentation. On the positive side, the hybrid approach benefits from several algorithmic and engineering advantages. Algorithmically, using YOLO's single-pass detector to guide

segmentation avoids the blind dense sampling required in TinySAM's hierarchical mode. This significantly reduces redundant computation and eliminates the multi-round refinement steps that slow down the original pipeline. The complementary use of sparse point prompts for background regions further ensures that both foreground and non-COCO objects are covered, enabling a more complete "segment everything" capability.

From an engineering standpoint, the modular structure of the hybrid pipeline makes it easy to optimize and debug. Each component, from YOLO for detection to TinySAM for segmentation, can be independently improved or swapped out. Both models rely on mature, well-documented open-source implementations. This contributes to the strong reproducibility of our experiments and lowers the barrier for others to extend or deploy the system.

However, the hybrid system also has notable limitations that stem primarily from the detector. As mentioned, for Detector-based objects, the YOLO→TinySAM pipeline achieves 40.7% AP, which is lower than the 46.7% achieved by ViTDet→TinySAM. On the full COCO dataset of 5000 images, this issue becomes more pronounced, with AP dropping to 10.7%. Additionally, direct comparisons with the hierarchical baseline can be misleading. For example, in images containing only large objects, YOLO tends to generate holistic masks (e.g., an entire person), while the hierarchical method often produces fine-grained part-level masks such as clothing or accessories. These reflect different notions of segmentation granularity, making a direct comparison difficult.

Overall, the hybrid model provides substantial gains in speed and coverage, but its precision remains bounded by detector performance. These findings suggest that future improvements should focus on boosting YOLO's recall, especially for small or uncommon categories, or incorporating additional cues—such as adaptive sampling or semantic post-processing—to mitigate detector omissions.

Future Work

1. **Fine-tune YOLO**: Fine-tune YOLOv12 on COCO dataset to improve detection rates for low-frequency categories and small objects, expected to boost AP to 45%+;
2. **Adaptive Grid Density**: Dynamically adjust sparse point sampling based on YOLO coverage density;
3. **Category-aware Post-processing**: Use CLIP to unify segmentation granularity for fairer comparison;
4. **Video Extension**: Leverage temporal information to improve segmentation consistency.

Conclusion

In this paper, we successfully developed and evaluated Tiny-YOLOSAM, a hybrid pipeline that combines the speedy object detection of YOLOv12 with the robust segmentation capabilities of TinySAM. Our experiments demonstrated that this approach significantly outperforms TinySAM's native hierarchical segment everything mode, achieving a 4.7x speedup while significantly improving segmentation quality and coverage, dramatically increasing *AR* from 16.4% to 77.1%. By taking advantage of YOLO for foreground objects and supplementing with a sparse point grid for background context, our method provides a balanced and efficient solution for generating complete scene segmentation.

Our findings present a viable pathway for achieving fast and high-quality image segmentation on consumer-grade hardware, bridging the gap between large, computationally expensive foundational models and practical applications. While our hybrid model shows a tradeoff in *AP* due to the detector's recall limits, future work will focus on mitigating this by finetuning YOLOv12 to improve detection of smaller and/or unusual objects. In addition, evaluating the hybrid system on a broader range of datasets would help establish its generality beyond COCO and better position it for real-world deployment.

With our hybridization of YOLOv12 and TinySAM, the segment everything capability originally offered by TinySAM is now far more accessible. We now have a model that is extremely fast and lightweight, while still preserving the strong segmentation performance of TinySAM. Both of these models are also easily available and already pretrained, which offers robust support with a simple API. With that, we can significantly lower the barrier to entry for quality segmentation and expand the societal and practical reach of segment-everything systems.

Societal Impact

This development represents another step forward toward democratizing access to vision technology and AI. However, broadened access also enables potential misuse, leading to additional risks and negative consequences. In particular, segment-everything capabilities lend themselves to being used for purposes such as surveillance, unauthorized monitoring, and facial/body segmentation in sensitive contexts. With improved access also comes the risk of over-reliance and use without adequate validation. It will be important for future work and future regulation to consider not only where this technology can be applied, but where it should and should not be applied, and to encourage responsible deployment and appropriate safeguards.

References

Kirillov, A.; Mintun, E.; Ravi, N.; Mao, H.; Rolland, C.; Gustafson, L.; Xiao, T.; Whitehead, S.; Berg, A. C.; Lo, W.-Y.; Dollár, P.; and Girshick, R. 2023. Segment Anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 4015–4026.

Lin, T.-Y.; Maire, M.; Belongie, S.; Hays, J.; Perona, P.; Ramanan, D.; Dollár, P.; and Zitnick, C. L. 2014. Microsoft COCO: Common Objects in Context. In *European Conference on Computer Vision (ECCV)*, 740–755. Springer.

Shu, H.; Li, W.; Tang, Y.; Zhang, Y.; Chen, Y.; Li, H.; Wang, Y.; and Chen, X. 2023. TinySAM: Pushing the Envelope for Efficient Segment Anything Model. *arXiv preprint arXiv:2312.13789*.

Yunjie, T.; Qixiang, Y.; and David, D. 2024. YOLOv12: Attention-Centric Real-Time Object Detectors. https://github.com/sunsmarterjie/yolov12. Accessed: 2024-12-01.

Zhang, C.; Han, D.; Qiao, Y.; Kim, J. U.; Bae, S.-H.; Lee, S.; and Hong, C. S. 2023a. MobileSAMv2: Faster Segment Anything to Everything. *arXiv preprint arXiv:2312.09579*.

Zhang, C.; Han, Z.; Shen, Y.; Wang, Y.; Qiao, S.; Liu, Z.; Liu, Y.; Li, X.; Guo, X.; and Liu, X. 2023b. EdgeSAM: Prompt-In-the-Loop Distillation for On-Device Deployment of SAM. *arXiv preprint arXiv:2312.06660*.

Zhao, X.; Ding, W.; An, Y.; Du, Y.; Yu, T.; Li, M.; Tang, M.; and Wang, J. 2023. Fast Segment Anything. *arXiv preprint arXiv:2306.12156*.

Individual Contributions

Kenneth Xu: Replicated the TinySAM model alongside Mason Wu and led the design of the hybrid extension. Connected YOLO and TinySAM within the unified pipeline, implemented key components of the codebase, and produced the main method diagrams and visualizations. Coordinated group meetings and meetings with the instructor to guide the project's direction.

Mason Wu: Replicated TinySAM model with Kenneth Xu, evaluated on 5000 images COCO dataset. Created demo with evaluation for current extension (Demo Repository: https://github.com/498ers/TinySAM). Added batch processing to current extension, ran through current repo's evaluation and validated metrics improvement. Wrote the Background, baseline of Methodology, Results, and Discussion; Extensions parts of the paper. Provided tables and Appendix H, I in paper.

Abby Schultz: Replicated graphs comparing YOLOv12 to several previous versions of YOLO, in terms of mAP vs latency and gFLOPs. Wrote the Abstract and Conclusion of the final paper.

Katherine Zhang: Helped reproduce the table and heatmap for YOLOv12 replication. Helped formulate TinySAM + YOLO extension, ensured consistent YOLO output. Gathered relevant work and wrote introduction, related work, and societal impact sections for final paper.

Max Han: Provided paper summaries, replicated YOLOV12, tested YOLOV12 + tinySAM pipeline, performed hyper-parameter tuning, evaluated its performance, and wrote YOLOV12 replication and YOLO appendix.

APPENDIX

Appendix A: Hard Mining Full-Stage Knowledge Distillation

1. Students learn multiple levels of knowledge from the teacher: generating embedding, token, mask.

$$L_{embedding} = L(E^T_{img}(I), E^S_{img}(I))$$

$$L_{output} = L(D^T_{mask}(E^T_{img}(I), q), D^S_{mask}(E^S_{img}(I), q))$$

$$L_{output} = L(T^T_{mask}(E^T_{img}(I), q), T^S_{mask}(E^S_{img}(I), q))$$

$$L_{distill} = L_{embedding} + L_{token} + L_{output}$$

$$L_{total} = L_{distill} + L_{mask} + L_{ious}$$

2. Hard Mask Weighting - puts more weight on complicated instances, which makes the learning process focus on more complicated patterns.

$$H_i = sigmoid(\frac{IoU(M^T_i, MiGT)}{IoU(M^S_i, M^{GT}_i)+\epsilon} - 1)$$

$$L_{distill} = \alpha L_{embedding} + \beta L_{token} + \gamma \sum_{i=1}^{N} H_i L^i_{output}$$

3. Hard Prompt Sampling - makes training samples concentrate in the difficult area for prediction. Points P_0 are initially sampled inside the labeled mask region M_{gt}. This initial points are fed into the network with input image to get the predicted mask region M_0. Then the next point is sampled such that $P_{i+1} = M_{gt} - M_i, i = 0, 1, 2, 3...$ With more iterations, the sampled dots get close to the edge of the ground true mask.

Appendix B: Quantization

The goal is to project floating point tensor x to b-bit integer tensor x_q with a scaling factor s. $O = AB => O' = A'B'$ A and B are transformed A' and B' by scaling factor SA and SB. Using $L = KL(y'_{pred}, y_{pred}) + KL(y'_{iou}, y_{iou})$, it find optimal values for SA and SB in the ranges of $[Q_l \frac{A_{max}}{2_{b-1}}, Q_u \frac{A_{max}}{2^{b-1}}]$ and $[Q_l \frac{B_{max}}{2^{b-1}}, Q_u \frac{B_{max}}{2^{b-1}}]$ respectively.

Appendix C: Hierarchical Segment-Everything

Sam samples points as a grid to segment everything, which leads to over-fine-grained segmentation results and also occupies massive computing resources. hierarchical segment-everything addresses this issue. For the first step, it uses only ¼ points on each side so the total points is 1/16 of the original to predict masks. Masks with confidence exceeding threshold T are considered final predictions. And it samples again as the same density with the original setting but ignores points in the predicted area.

Appendix D: Area Attention

Area attention module maintains a large receptive field while reducing the computational complexity of attention in a very simple way, thus enhancing speed. A feature map of size (H, W) is divided into l segments of size $(\frac{H}{l}, W)$ or $(H, \frac{W}{l})$. In the paper, $l = 4$, which reduces the computational cost from $2n^2hd$ to $\frac{1}{2}n^2hd$.

Appendix E: Flash Attention

Flash Attention conquers the memory access issue of attention, removing designs such as positional encoding to make the model fast and clean, adjusting the MLP ratio from 4 to 1.2 to balance the computational between attention and feed forward network for better performance, reducing the depth of stacked blocks to facilitate optimization, and making use of convolution operators as much as possible to leverage their computational efficiency.

Appendix F: Residual Efficient Layer Aggregation Network

Residual efficient layer aggregation networks (R-ELAN) to address the optimization challenges introduced by attention, which improved the original ELAN. Efficient layer aggregation networks (ELAN) causes gradient blocking and lacks residual connection from input to output.
Residual Efficient Layer Aggregation Network

1. Applies block-level residual design with scaling techniques.

2. Applies a transition layer to adjust the channel dimensions and produces a single feature map.

3. This feature map is then processed through subsequent blocks followed by the concatenation, forming bottleneck structure.

4. This preserves the original feature integration capability, but also reduces both computational cost and parameter/memory usage.

Appendix G: YOLO Replication Graph

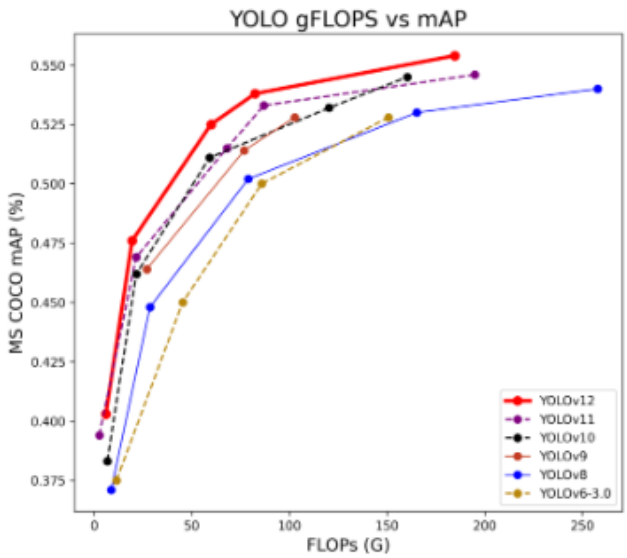


Figure 4: Accuracy–efficiency trade-off for object detection models on COCO. The plot shows mAP as a function of GFLOPs; YOLOv12 achieves higher mAP with the same computational cost than the previous YOLO models.

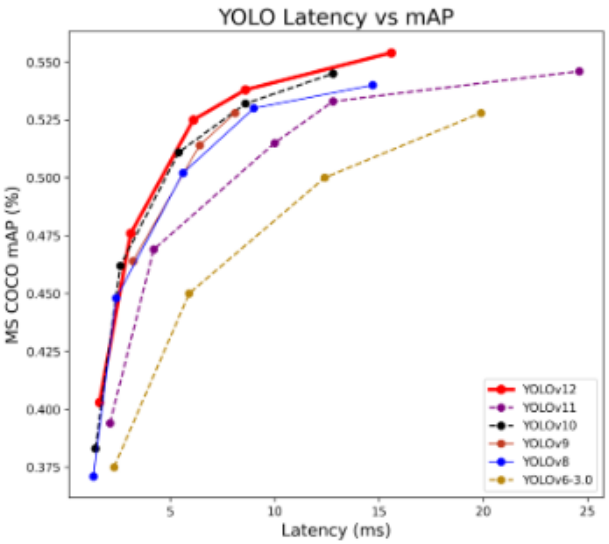
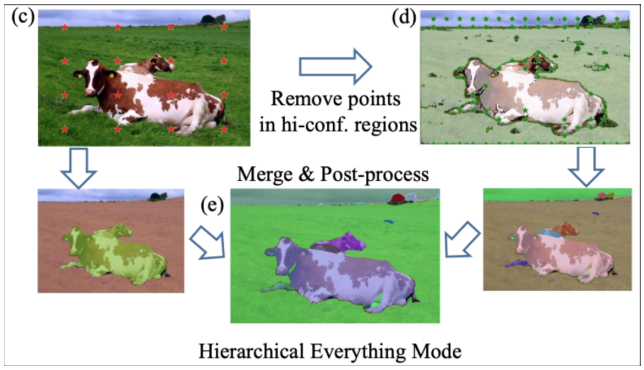


Figure 5: Latency–accuracy trade-off for all evaluated models on the COCO 2017 dataset. The plot shows mean Average Precision (mAP) as a function of inference latency; YOLOv12 achieves higher accuracy than the previous models while maintaining real-time performance.

Appendix H: Sampling Strategy Comparison



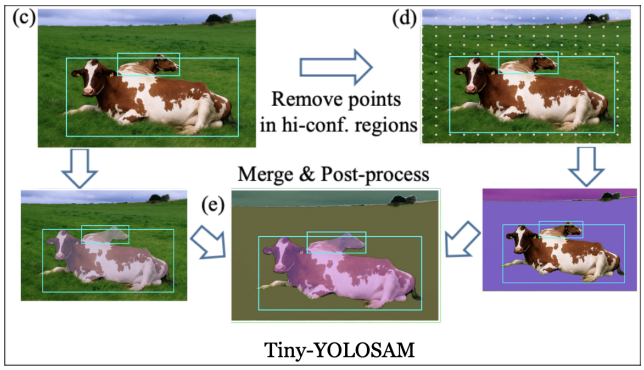


Figure 6: Hierarchical Everything Mode pipeline visualization v.s. Tiny-YOLOSAM pipeline visualization.

Both methods follow a similar workflow: (c) initial prompts are provided, (d) high-confidence regions are identified and removed from subsequent sampling, and (e) remaining areas are processed to generate final masks. The key difference lies in the prompting strategy: Hierarchical mode uses a dense 8×8 point grid covering the entire image and performs adaptive resampling in low-confidence areas, resulting in fragmented masks. In contrast, Tiny-YOLOSAM leverages YOLO-detected bounding boxes to segment foreground objects and applies sparse grid sampling only to uncovered background regions, producing more complete and coherent scene segmentation.

Appendix I: Quantitative Performance Comparison

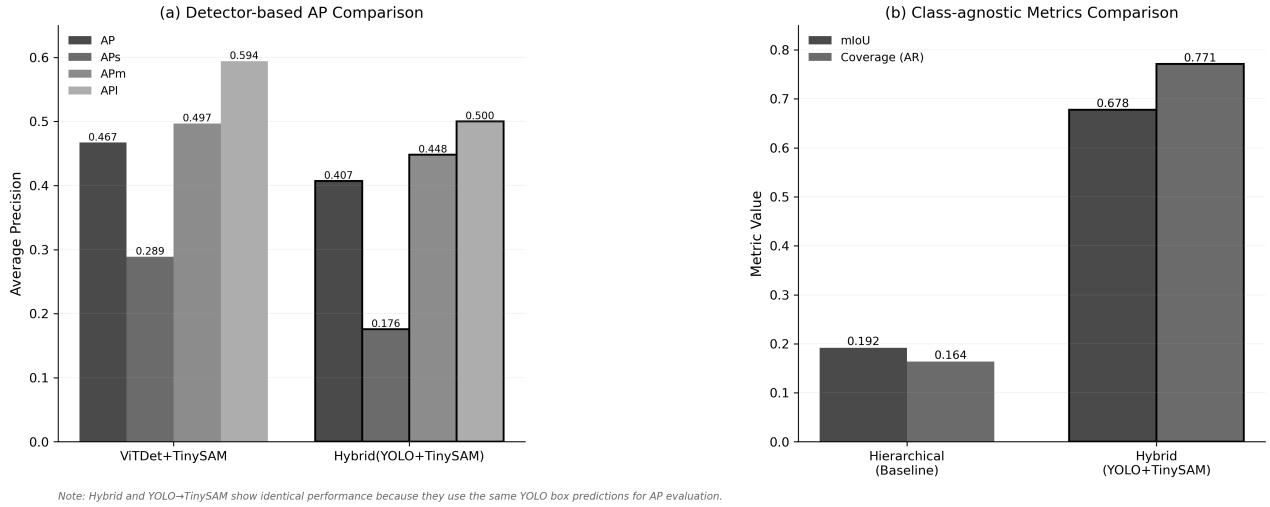


Figure 7: Performance comparison.

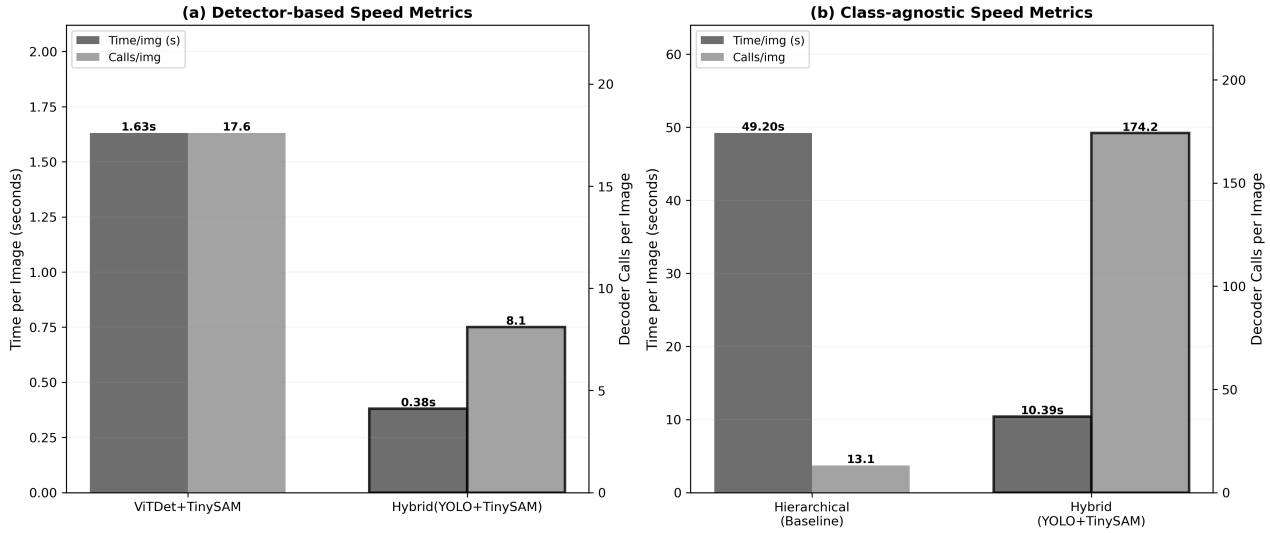


Figure 8: Speed comparison.